Runtime Analysis and Introduction to Sets

Yu-Shan Sun

Department of Computer Science
Worcester Polytechnic Institute
Worcester, MA 01609 USA

CS2102-B24-BL02: Tuesday, Nov. 26, 2024

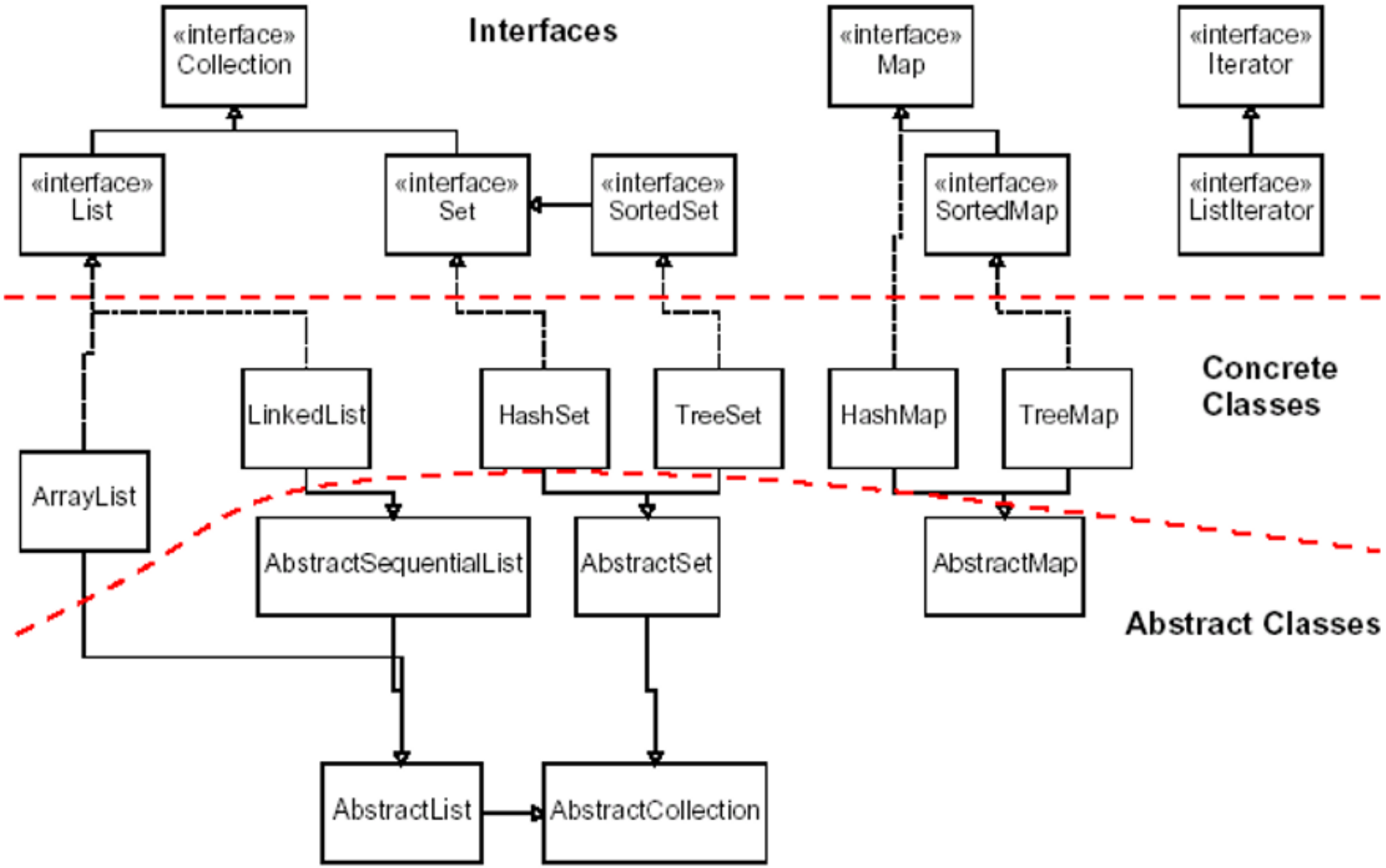
Slide Sources:

1. *Building Java Programs* by Stuart Reges and Marty Stepp
2. Previous/Current CS 2102 Instructors @ WPI
3. Previous/Current CPSC 1010/1060 Instructors @ Clemson University
4. Previous/Current CPSC 2150 Instructors @ Clemson University
5. Prof. Kevin Wayne @ Princeton University (COS 226 – Spring 2014)
6. The lecture slides have also benefitted from instructors at Ohio State: Paolo Bucci, Wayne Heym, Paul Sivilotti, and Bruce Weide.

Collections
(REVIEW)

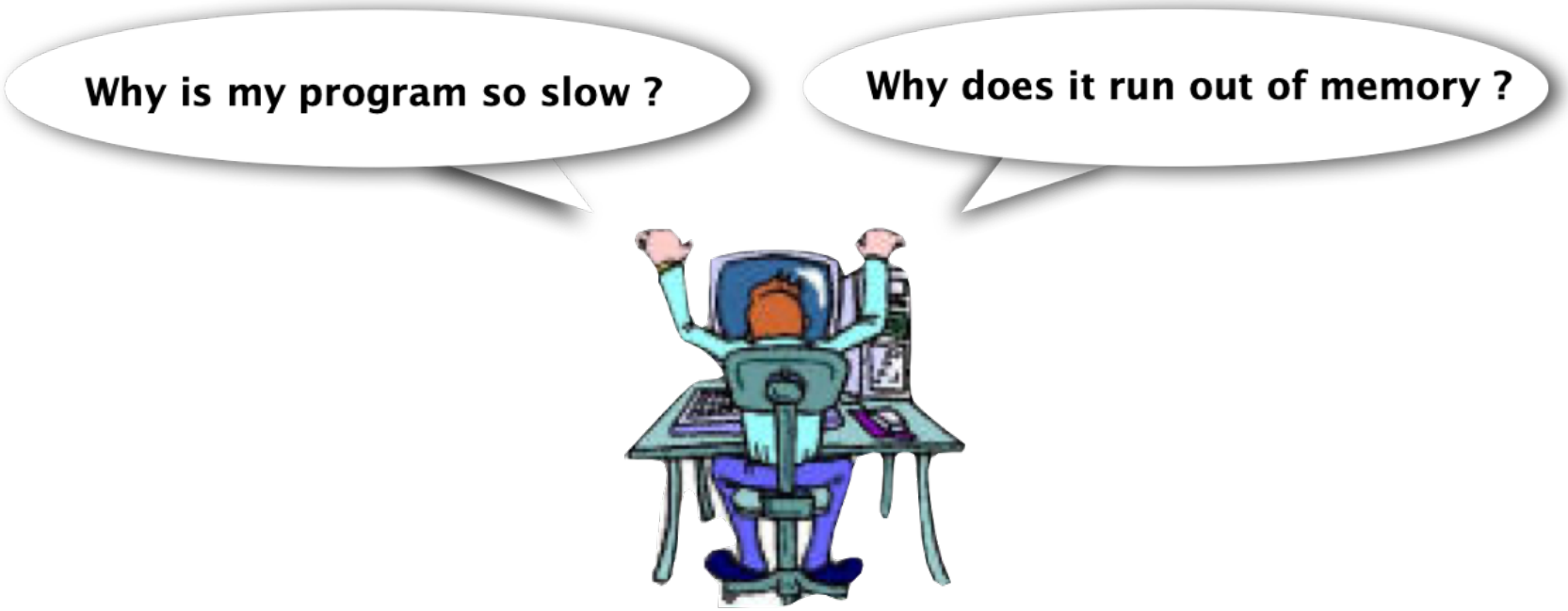
- **Collection:** an object that stores data
  - the objects stored are called **elements**
  - some collections maintain an ordering; some allow duplicates
  - **typical operations:** `add`, `remove`, `clear`, `contains` (search), `size`
  - **examples found in the Java class libraries:** `ArrayList`, `LinkedList`, `HashMap`, `TreeSet`, `PriorityQueue`
  - all collections are in the `java.util` package
    `import java.util.*;`

Java Collections Framework
(REVIEW)



The Challenge

Will my program be able to solve a large practical input?



Observations

- **How long will my program take?**
  - **Problem size:** size of input or value of a command-line argument
- **Intuitively:** Running time increases with problem size
  - . . . BUT by how much?

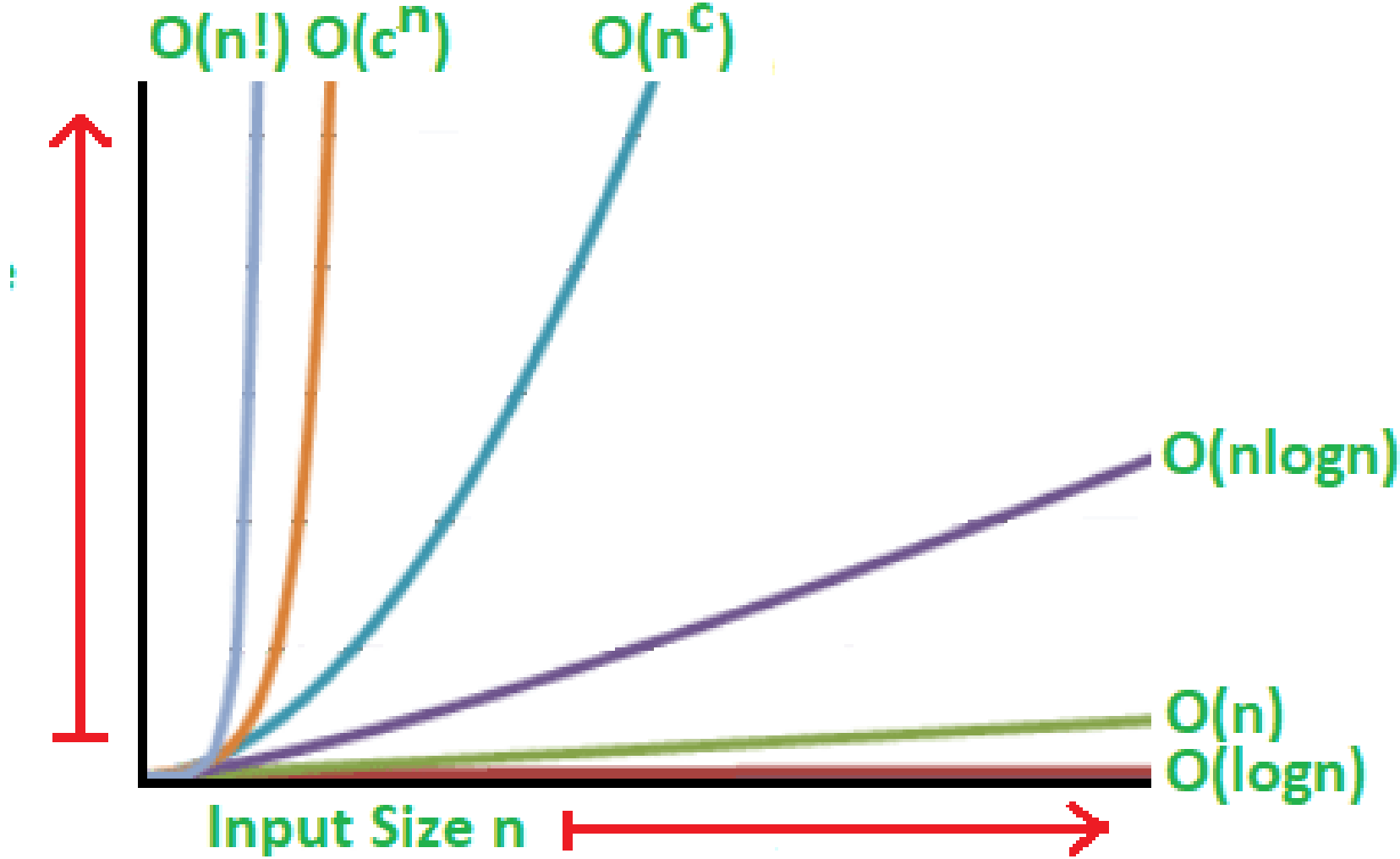
Types of Analysis

- Let's focus our analysis on the **worst case**
  - Determined by "most difficult input"
  - Provides a guarantee for all inputs

Big-$\mathcal{O}$ Notation

- **Big-$\mathcal{O}$** notation is used to develop upper bounds for algorithm analysis
  - **Upper bound:** Performance guarantee of the algorithm for any input.
- **Example 1:** Adding to the end of a linked-list is $\mathcal{O}(1)$
- **Example 2:** Looping through a list is $\mathcal{O}(N)$, where N here refers to the number of elements in my list.
- You will learn more when you take **CS 2223: Algorithms**

Common Encountered Order-Of-Growth Functions

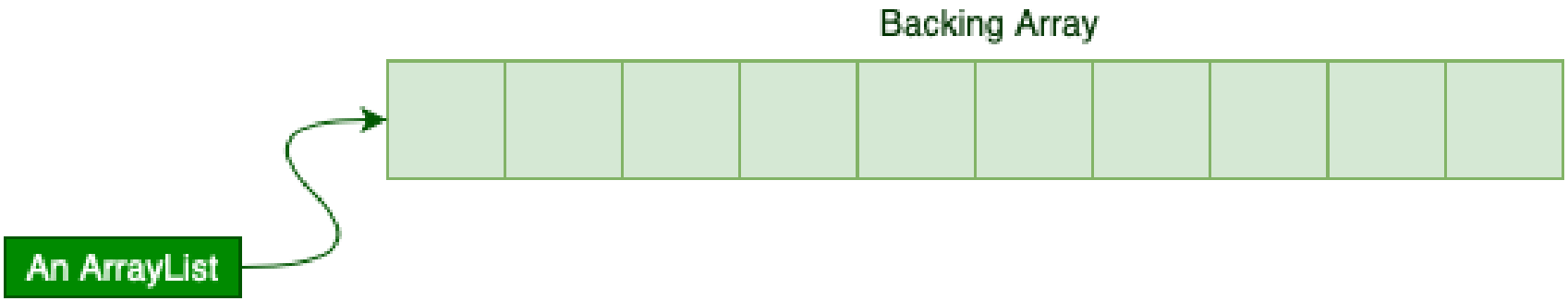


ADTs: Lists

Basic List Operations

- **add:** Adds element to the end of the list
- **get:** Examines an element from a specific index but doesn't remove it
- **remove:** Removes from a specific index in the list
- **contains:** Searches for an element in the list
- **isEmpty:** Checks if the list is empty
- **size:** The amount of elements in the list

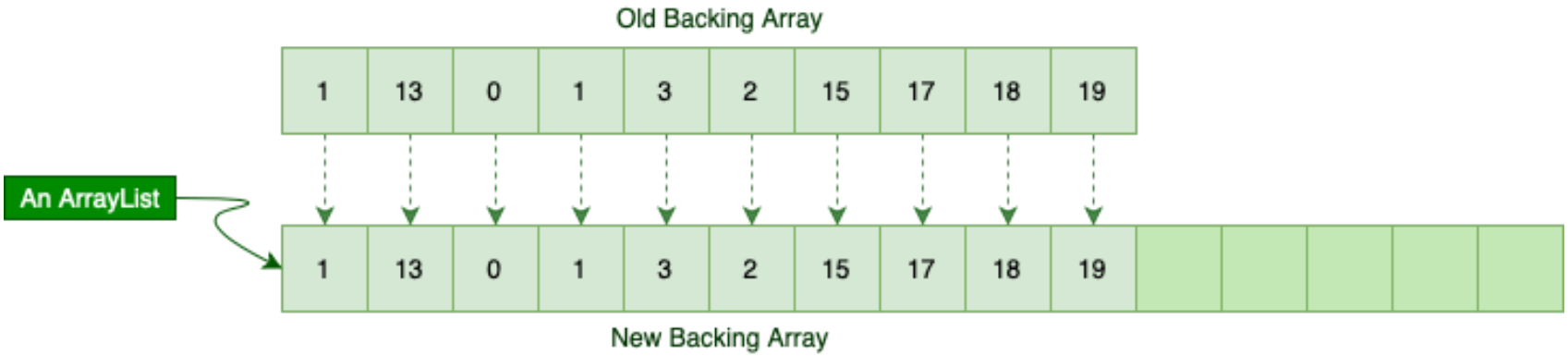
Illustration of Java's `ArrayList`[1]



Array Implementation

[1] (**Src:** https://www.baeldung.com/java-arraylist-linkedlist)

Illustration of Java's `ArrayList` Add Method[2]



Automatically Resizes When Running Out of Room

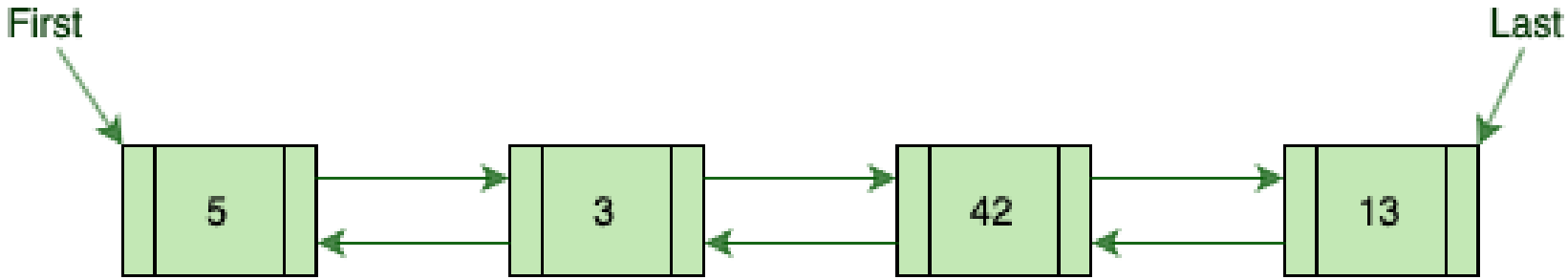
[2] (**Src:** https://www.baeldung.com/java-arraylist-linkedlist)

Illustration of Java's `ArrayList` Remove Method[3]

Shift Items to Eliminate Gaps

[3] (**Src:** https://www.baeldung.com/java-arraylist-linkedlist)

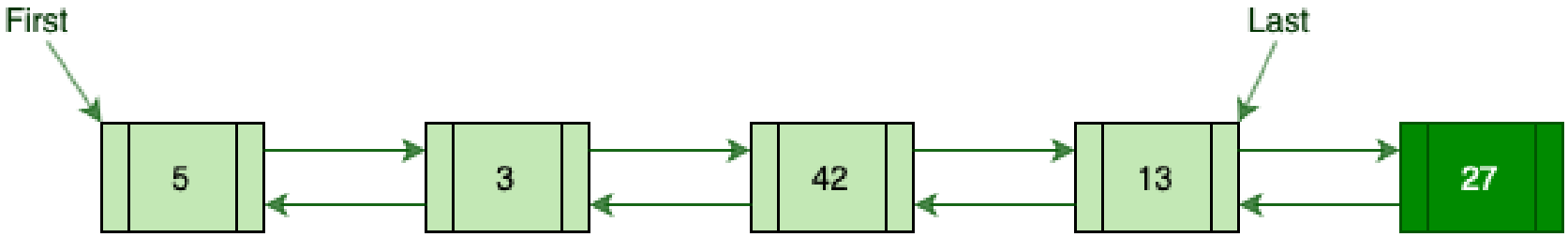
Illustration of Java's `LinkedList`[4]



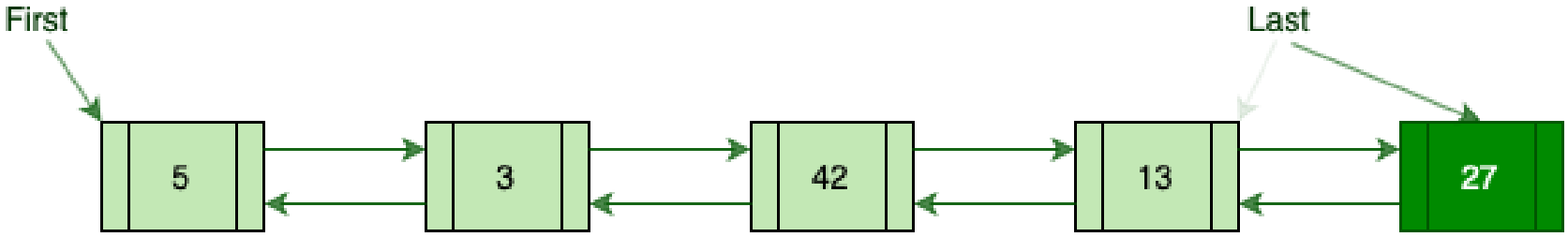
Can be Traversed in Either Direction

[4] (**Src:** https://www.baeldung.com/java-arraylist-linkedlist)

Illustration of Java's `LinkedList` Add Method[5]



Step 1: Add Element to End

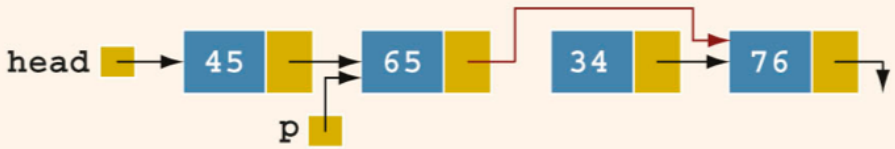


Step 2: Update the Reference to Last

[5] (**Src:** https://www.baeldung.com/java-arraylist-linkedlist)

Illustration of Java's `LinkedList` Remove Method

Step 1: Find the Item in List by Traversing



Step 2: Reattach Chain and Remove Item

Comparing ArrayList and LinkedList

Runtime

Operation	Runtime	
	ArrayList	LinkedList
add	$\mathcal{O}(1)$[6]	$\mathcal{O}(1)$
get	$\mathcal{O}(1)$	$\mathcal{O}(N)$
remove	$\mathcal{O}(N)$	$\mathcal{O}(N)$
contains	$\mathcal{O}(N)$	$\mathcal{O}(N)$
isEmpty	$\mathcal{O}(1)$	$\mathcal{O}(1)$
size	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Finding minimum	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Finding maximum	$\mathcal{O}(N)$	$\mathcal{O}(N)$

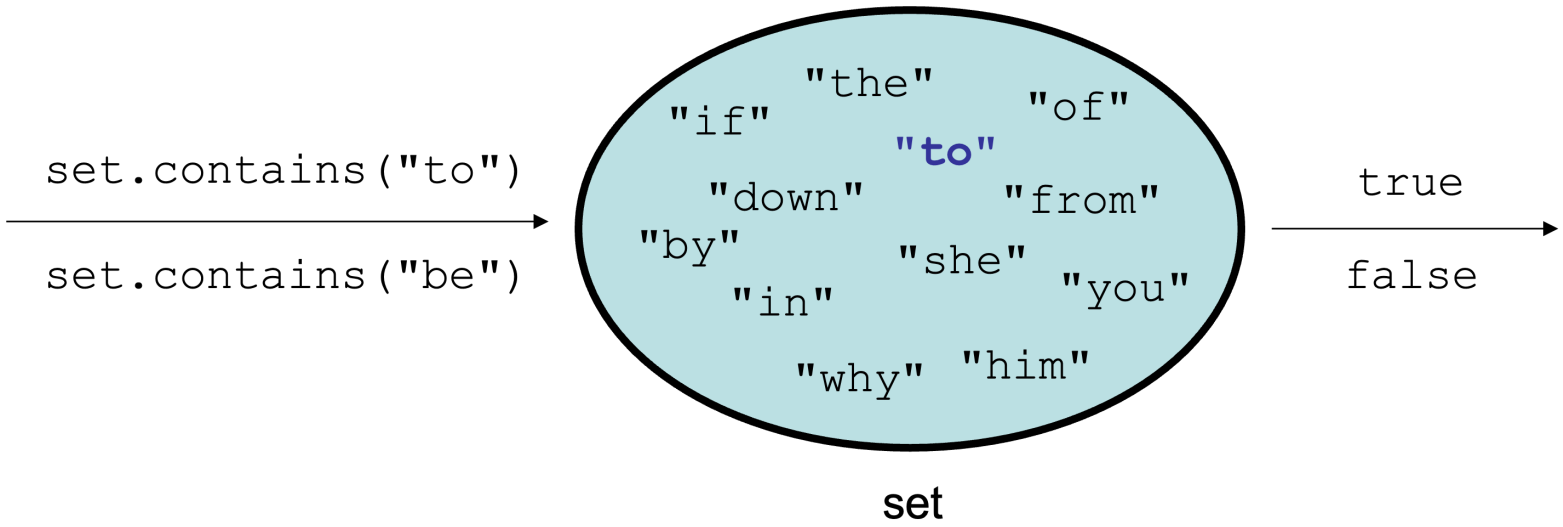
[6] Amortized cost when we don't need to resize

Another ADT

Is there an ADT similar to `List`, but doesn't allow for duplicates?

Set

- **Set**: A collection of unique values (no duplicates allowed) that can perform the following operations efficiently:
  - add, remove, search (contains)
- We don't think of a set as having indexes; we add things to the set in general and don't worry about the order



- In Java, sets are represented by `Set` interface in `java.util`

`Set` Implementations

- There are 3 implementations of `Set`
  1. `HashSet`: implemented using a "hash table" array[7]; very fast: $\mathcal{O}(1)$ for all operations, but elements are stored in unpredictable order
  2. `TreeSet`: implemented using a "binary search tree"; pretty fast: $\mathcal{O}(log N)$ for all operations, and elements are stored in sorted order
  3. `LinkedHashSet`: $\mathcal{O}(1)$ but stores in order of insertion

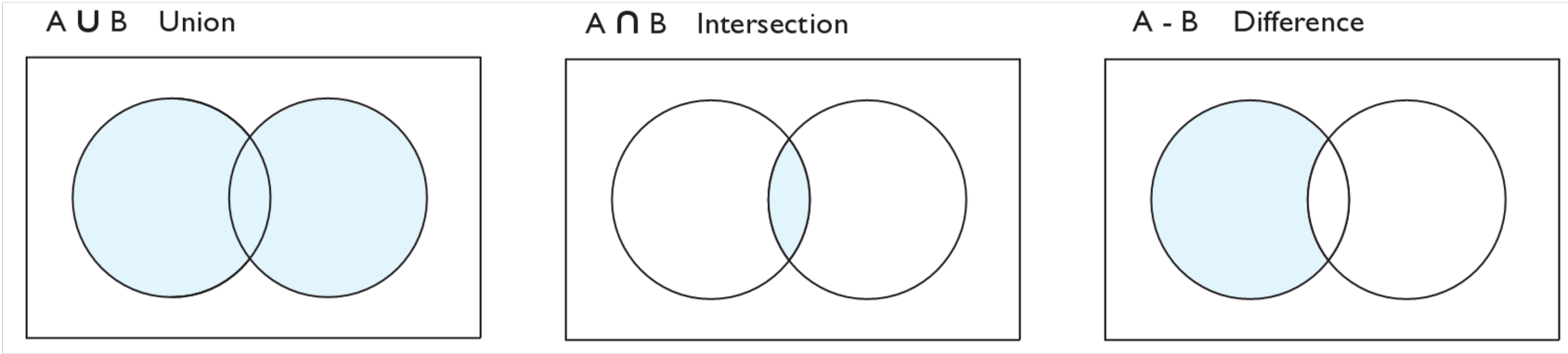
[7] We will talk about hashing in a future lecture, but just know lookup by equality (rather than index) is fast

Comparing HashSet, TreeSet and LinkedHashSet

Runtime

Operation	Runtime		
	HashSet	TreeSet	LinkedHashSet
add	$\mathcal{O}(1)$	$\mathcal{O}(logN)$	$\mathcal{O}(1)$
remove	$\mathcal{O}(1)$	$\mathcal{O}(logN)$	$\mathcal{O}(1)$
contains	$\mathcal{O}(1)$	$\mathcal{O}(logN)$	$\mathcal{O}(1)$
isEmpty	$\mathcal{O}(1)$	$\mathcal{O}(logN)$	$\mathcal{O}(1)$
size	$\mathcal{O}(1)$	$\mathcal{O}(logN)$	$\mathcal{O}(1)$
Finding minimum	$\mathcal{O}(N)$	$\mathcal{O}(logN)$	$\mathcal{O}(N)$
Finding maximum	$\mathcal{O}(N)$	$\mathcal{O}(logN)$	$\mathcal{O}(N)$

Other Set Operations



`addAll(`**collection**`)`	adds all elements from the given collection to this set
`containsAll(`**coll**`)`	returns `true` if this set contains every element from given set
`equals(`**set**`)`	returns `true` if given other set contains the same elements
`iterator()`	returns an object used to examine set's contents *(seen later)*
`removeAll(`**coll**`)`	removes all elements in the given collection from this set
`retainAll(`**coll**`)`	removes elements *not* found in given collection from this set
`toArray()`	returns an array of the elements in this set

In-Class Example: Example Usage of `Sets`

Unique words in "A Tale of Two Cities" by Charles Dickens and in "Aesop's Fable"

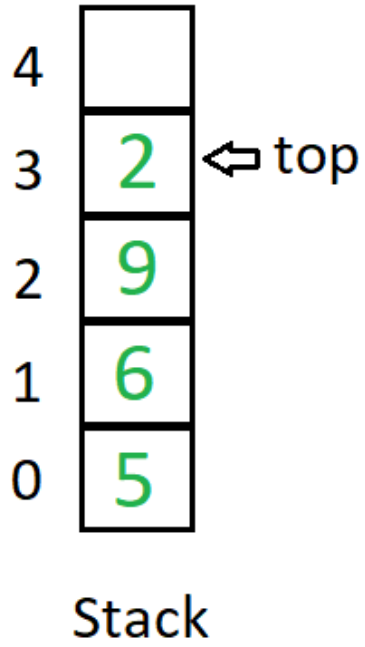
ADTs: Stacks and Queues

- Sometimes it is good to have a collection that is less powerful, but is optimized to perform certain operations very quickly.
- We will examine two specialty collections:
  1. **Stack:** Retrieves elements in the reverse of the order they were added.
  2. **Queue:** Retrieves elements in the same order they were added.

ADTs: Stacks

Basic Stack Operations

- **push:** Adds element at the top
- **pop:** Removes the top element
- **peek:** Examines the top element but doesn't remove it
- **isEmpty:** Checks if the stack is empty
- **depth:** The amount of elements in the stack
- **clear:** Removes all elements in the stack



Stack Implemented Using LinkedList

Runtime

Operation	Runtime
push	$\mathcal{O}(1)$
pop	$\mathcal{O}(1)$
peek	$\mathcal{O}(1)$
contains	$\mathcal{O}(N)$
isEmpty	$\mathcal{O}(1)$
depth	$\mathcal{O}(1)$

ADTs: Queues

Basic Queue Operations

- **enqueue:** Adds element to the back
- **dequeue:** Removes the front element
- **peek:** Examines the front element but doesn't remove it
- **isEmpty:** Checks if the queue is empty
- **length:** The amount of elements in the queue
- **clear:** Removes all elements in the queue

Queue Implemented Using LinkedList

Runtime

Operation	Runtime
enqueue	$\mathcal{O}(1)$
dequeue	$\mathcal{O}(1)$
peek	$\mathcal{O}(1)$
contains	$\mathcal{O}(N)$
isEmpty	$\mathcal{O}(1)$
length	$\mathcal{O}(1)$

LLReversibleQueue Implemented Using LinkedLists

Runtime

Operation	Runtime	
	v1 (Like a Queue)	v3 (Using Two LinkedLists)
enqueue	???	???
dequeue	???	???
peek	???	???
reverse	???	???

Question: What is the trade-off in v3?

Summary

1. Big-$\mathcal{O}$ Notation and Order-of-Growth Functions
2. Abstract Data Type (ADT): Lists
3. Abstract Data Type (ADT): `Set`
   - Implementations
   - Operations
   - In-Class Example: Example Usage of `Set`s
4. Abstract Data Type (ADT)
   - Stacks
   - Queues